

DATA3404 Group Assignment 2

TUT07-AssignmentGroup-06

Task 1

The first task is to convert a piece of SQL code into Spark. In the original SQL, the data processing can be divided into the following four steps:

1. Use the host's "id" to join four tables together, and use "COUNT" to calculate how many unique reviews each ("listing_name" × "city_name") combination received.
2. Use four conditions in the "WHERE" clause to filter rows, selecting data where the host is a superhost, the listing is in Australia, the property type is "Entire home" or "apartment", and the year is 2025.
3. Group the data by "listing_name" and "city_name" using "GROUP BY".
4. Sort the result and output the top 5.
5. Next, we use PySpark DataFrame API to repeat the above steps.

We use Pyspark DataFrame API to repeat the above steps. First, we use the function "spark.read" to read external files in CSV format and let Spark infer the column types (int / double / string, etc.). This corresponds to the first step in SQL. Then, using the column "id", we perform "inner join" on the four tables one by one. After that, we use the "filter" to select rows where the host is a superhost, the year is 2025, the country is Australia, and the property type is "Entire home" or "apartment". Then, we group the data by "listing_name" and "city_name", remove duplicates, count the number of unique "review_id"s in each group, and aggregate the results. These steps will produce a new DataFrame. Finally, we use the "show" function to trigger the execution and display the result. The results for small, medium, and large datasets are as follows:

listing_name	city_name	num_reviews
One-Bedroom Apart...	Melbourne	53
South Brisbane Ci...	Brisbane	26
Wanderlust - I wa...	Melbourne	24
1br near airport ...	Sydney	22
Carlton Luxurious...	Melbourne	19

listing_name	city_name	num_reviews	listing_name	city_name	num_reviews
One-Bedroom Apart...	Melbourne	53	One-Bedroom Apart...	Melbourne	53
Brand New Studio ...	Melbourne	37	Blessington St St...	Melbourne	44
Lux Beach Retreat...	Sydney	30	Brand New Studio ...	Melbourne	37
Bayside Bungalow	Melbourne	25	Lux Beach Retreat...	Sydney	30
Miss Baker's Bond...	Sydney	25	St Kilda stunner ...	Melbourne	28

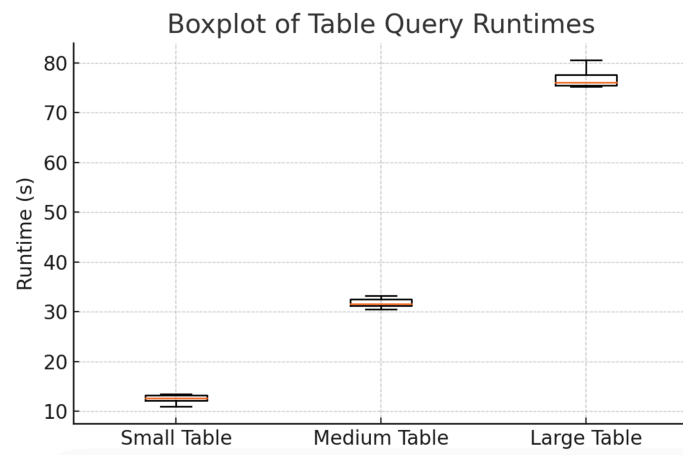
Query the output results on the small (top), medium (bottom-left), and large (bottom-right) datasets.

Next, in order to further measure the execution time for the small, medium, and large datasets, we run each query ten times and record the results in the table below. As shown, the query time increases

significantly as the dataset size grows. In the "large" dataset, the average query time has already reached over 76 seconds. Therefore, we will focus on optimizing the query for this dataset in the following steps.

Run Time Comparison for Small, Medium, large Table

Measurement#	Small Table Time (s)	Medium Table Time (s)	Large Table Time (s)
1	14.95	33.24	75.38
2	12.46	33.09	77.47
3	13.44	31.18	75.2
4	10.97	30.47	77.61
5	13.3	32.56	80.58
6	12.93	30.9	76.09
7	12.83	32.16	75.25
8	11.84	31.4	75.86
9	12.2	31.52	78.89
10	12.19	31.7	76.11
Mean	12.711	31.82	76.844



Task 2

In the previous task, we used Spark to replicate the exact same logic as the original SQL query, without applying any optimizations. We observed that the overall query time was quite long and the performance was suboptimal. Therefore, in this task, we aim to optimize the query both at the code level and the data structure level, and ultimately combine both approaches for improved performance.

We begin by optimizing the query logic through code adjustments, with the goal of reducing unnecessary I/O operations. Initially, the query followed this sequence: reading data → joining tables → filtering → computing → sorting. In this process, the join step combined all four datasets in full, including many columns that were never used in the final results. This led to excessive and unnecessary computation. To address this, we moved the filtering step earlier in the pipeline, applying it during the data reading phase. This ensures that only the necessary data is loaded and joined, which significantly reduces the volume of data involved in subsequent operations.

In addition, we noticed that the original code used a Sort-Merge Join strategy, which forced all four tables to be fully sorted before shuffling. This introduced two expensive "Exchange" stages and involved the transmission of multiple columns across the network. To improve on this, we switched to a Broadcast Hash Join strategy. Specifically, we apply the "broadcast" hint only to the smaller tables—"Hosts" and "Cities"—after they have been filtered down to under 10 MB in size. This allows the much larger "Listings" table to perform a local hash join within each partition. By broadcasting the small tables from the Driver to all Executors in memory, we eliminate the need for network shuffling or repeated data transfers during the join.

The differences in the optimized physical plan are summarized below:

```
:- *(4) Project [listing_name#634, city_id#640,
id#686L]
      : +- *(4) BroadcastHashJoin [id#633L],
[listing_id#687L], Inner, BuildRight, false, false
```

Taking the "Listings × Reviews" branch as an example, we can observe that each table performs a "Project" operation immediately after the "FileScan", retaining only the join keys and the required output columns before entering the "BroadcastHashJoin". This confirms that we first prune the columns we need, and then proceed with the join and broadcasting.

Moreover, after explicitly specifying the join type as "broadcast", the join between "Listings" and "Reviews" switches from a Sort-Merge Join to a Broadcast Hash Join. This eliminates the need for a global shuffle and dual-side sorting. Combined with column pruning, this significantly reduces network load and improves overall performance.

```
      : +- InMemoryTableScan [id#686L,
listing_id#687L], [isnotnull(listing_id#687L)], false
      : +- InMemoryRelation [id#686L,
listing_id#687L], StorageLevel(disk, memory, deserialized, 1 replicas)
```

The result shown in the diagram indicates that the reduced subset of the "Reviews" table has been

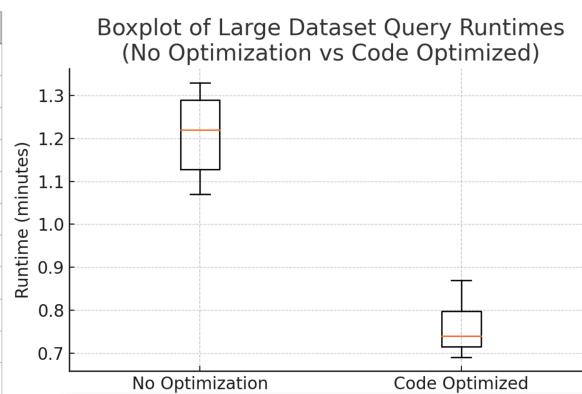
cached. Subsequent computations read directly from memory instead of re-executing filters or scanning the disk again. This avoids the overhead of reapplying filters and projections. After caching, only a small amount of data (a few dozen KB) needs to be copied across the network to each Executor in order to complete the broadcast.

```
+-- *(6) HashAggregate(keys=[listing_name#634, city_name#716],
functions=[finalmerge_count(distinct merge count#1133L) AS
count(id#686L)#1099L], output=[listing_name#634, city_name#716,
num_reviews#1098L])
+-- Exchange hashpartitioning(listing_name#634, city_name#716, 200),
ENSURE_REQUIREMENTS, [plan_id=2283]
+-- *(5) HashAggregate(keys=[listing_name#634, city_name#716],
functions=[partial_count(distinct id#686L) AS count#1133L],
output=[listing_name#634, city_name#716, count#1133L])
+-- *(5) HashAggregate(keys=[listing_name#634, city_name#716,
id#686L], functions=[], output=[listing_name#634, city_name#716, id#686L])
+-- Exchange hashpartitioning(listing_name#634, city_name#716,
id#686L, 200), ENSURE_REQUIREMENTS, [plan_id=2278]
+-- *(4) HashAggregate(keys=[listing_name#634, city_name#716,
id#686L], functions=[], output=[listing_name#634, city_name#716, id#686L])
```

As a result of replacing the previous Sort-Merge Join with a Broadcast Join, one pipeline stage was eliminated. The Shuffle stage associated with the Sort-Merge Join was also removed, leading to a significant reduction in execution time and network I/O.

After running the optimized query on the large dataset ten times and comparing it with the unoptimized version, it is clear that the optimized query performs much better. The average query time has been reduced to just over 20 seconds.

Measure#	No Optimization	Code Optimized
1	1.25	0.69
2	1.29	0.71
3	1.08	0.79
4	1.33	0.82
5	1.32	0.75
6	1.07	0.73
7	1.11	0.69
8	1.29	0.73
9	1.19	0.80
10	1.18	0.87
Mean	1.21	0.76



Next, we focus on optimizing the query from the physical layer. This means we do not change the PySpark query logic, but instead improve performance by modifying the storage format or layout, allowing the engine to read data faster and scan less. We observe that the original files are in CSV format. Directly processing CSV files often introduces severe performance bottlenecks. For example, every read requires row-by-row data type parsing, columns cannot be efficiently skipped, and there is no statistical metadata available. As a result, queries consume excessive CPU and I/O resources. Through physical optimization, we can leverage advanced storage formats such as Parquet, which provide columnar storage. This significantly reduces I/O overhead and improves overall execution efficiency. In addition, physical optimization lowers the cost of code maintenance

and increases query stability. Therefore, before executing the queries, we convert the original CSV files to Parquet format. We then apply the same optimized query steps as before.

Below are the key differences in the physical plan after applying physical optimization:

```

      +- *(2) Filter isnotnull(listing_id#1571)
      +- *(2) ColumnarToRow
      +- FileScan parquet
[id#1570,listing_id#1571] Batched: true, DataFilters: |
[isnotnull(listing_id#1571)], Format: Parquet, Location:
InMemoryFileIndex(1
paths) [dbfs:/FileStore/tables_parquet/airbnb_reviews_by_year.parquet/yr=202
5], PartitionFilters: [], PushedFilters: [IsNotNull(listing_id)],

```

Batched: true indicates that Spark is using a vectorized reader, which decodes thousands of rows at a time into a columnar batch buffer. This is several times faster than parsing CSV files row by row.

ColumnarToRow is a lightweight physical operator that converts columnar batches into row format, which is required by downstream operations such as HashJoin or Aggregate. This further reduces overall CPU time.

In the diagram, ColumnarToRow appears above FileScan Parquet due to the way the physical plan is printed. However, in the actual execution flow, it works as follows: Spark performs a vectorized scan of Parquet files (Batched: true) → reads data into a ColumnBatch → then ColumnarToRow converts the column batch into rows, which are then consumed by the upper-level BroadcastHashJoin.

The path “dbfs:/FileStore/tables_parquet/airbnb_reviews_by_year.parquet/yr=2025” indicates that only the subdirectory for yr=2025 is read. This means Spark does not scan the entire table and then filter the data—instead, it reads only the relevant partition directly.

```

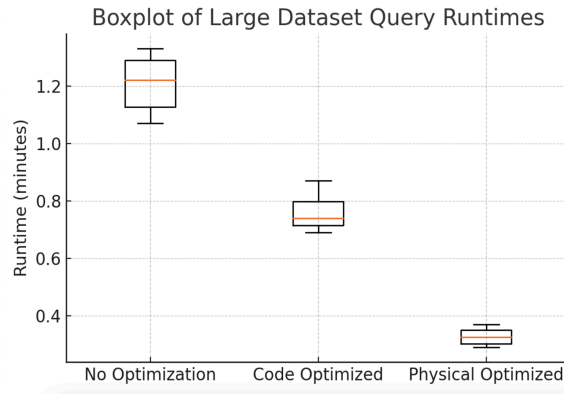
      +- *(4) BroadcastHashJoin [city_id#1541], [id#1524],
      Inner, BuildRight, false, false|

```

Here, we can see that after applying physical optimization, the knownFloatingPointNormalized operation is removed. This means that the original normalization function is no longer present, reducing the overhead of serialization and deserialization. Please refer to the appendix B for the Execution Plan(Page14-16).

Finally, when we compare the unoptimized, code-optimized, and physically optimized versions side by side, we can clearly observe the differences:

Measure#	No Optimization	Code Optimized	Physical Optimized
1	1.25	0.69	0.35
2	1.29	0.71	0.3
3	1.08	0.79	0.31
4	1.33	0.82	0.37
5	1.32	0.75	0.33
6	1.07	0.73	0.3
7	1.11	0.69	0.36
8	1.29	0.73	0.35
9	1.19	0.80	0.29
10	1.18	0.87	0.32
Mean	1.21	0.76	0.328



From this, we can definitely conclude that the effect of physical optimization is the best. To a large extent, it reduces the running time and computing cost.

Metric	No Optimization	Code Optimized	Physical Optimized
Elapsed Time (min)	1.20	0.75 (-38%)	0.33 (-73%)
Executor Run Time (min)	7.20	3.90 (-46%)	1.20 (-83%)
GC Time (s)	9.0	7.0 (-22%)	2.0 (-78%)
Shuffle Write / Read (MB)	11.4 / 11.4	11.4 / 11.4	7.3 / 7.3
Task Count	642	619 (-4%)	416 (-35%)
Stage Count	15	7 (-53%)	10 (-33%)

Similarly, we can more clearly see the differences in different degrees of optimization on this Stage Matrix. Compared with ordinary queries, the running time of physical optimization is reduced by 73%. A detailed introduction to StageMatrix will be provided in Task3.

Task 3

Solution approach

In this analysis, we perform query operations on four tables: cities, neighbourhoods, reviews, and listings. The goal is to analyze Airbnb listing data for a specific city (e.g., Sydney) in the year 2024, identify the top 10 neighborhoods with the highest occupancy rates, and within each of those neighborhoods, find the top 5 listings with the highest occupancy rates.

We begin by filtering listings for the specified city and joining with the neighborhood names. We then filter out the review records for the year 2024 and join them as well. Next, calculate the "number of occupied days" for each review. This is done by computing the length of stay for each review based on the listing's "minimum_nights". If the "minimum_nights" value is between 4 and 21, we use it as-is; otherwise, we default to 3 nights.

Assuming a review rate of 50%—meaning each review corresponds to two actual stays—we multiply the stay length by 2 to get the number of occupied days contributed by each review. We then group the data by "listing_id" and sum up all the occupied days contributed by reviews, resulting in the total number of occupied days for each listing in the year (denoted as `occupation_days_sum`). To ensure that listings without reviews are also included in the analysis, we perform a left join between this aggregated result and the full listing table, treating missing values as 0 occupied days.

Finally, we divide the total occupied days by 365 and use `F.lit(1.0)` to cap the occupancy rate at 100%, producing each listing's annual occupancy rate. To clearly present the results, we group the listings by "neighbourhood_name" and compute the average occupancy rate for each neighborhood. We select the top 10 neighborhoods with the highest average occupancy rate. Within these neighborhoods, we sort listings by their occupancy rate and select the top 5 listings per neighborhood. The results are then formatted for output. Please refer to the appendix A for the final output result(Page13-14).

Optimization

We then proceed to optimize the entire query process through both Code Optimization and Physical Optimization.

Code Optimization This includes:

1. Selecting only the necessary columns when initially loading the tables, pushing down the projection to reduce the amount of data read.
2. Applying filters as early as possible to reduce unnecessary operations during joins, minimizing the amount of data carried forward in the pipeline.
3. Using broadcast joins instead of shuffle joins for small tables like cities and neighbourhoods, which avoids costly shuffles.
4. Caching frequently used intermediate results (such as the filtered reviews_2024) using the `cache()` function to avoid redundant computation.
5. Consolidating fragmented intermediate queries to avoid repeated joins and `groupBy` operations, thus reducing unnecessary intermediate DataFrames.

Physical Optimization

While code optimization improves logical efficiency, physical optimization enhances performance at the execution layer. We convert the original data storage format from CSV to Parquet, which is a columnar storage format well-suited for analytical queries due to better compression and faster read performance.

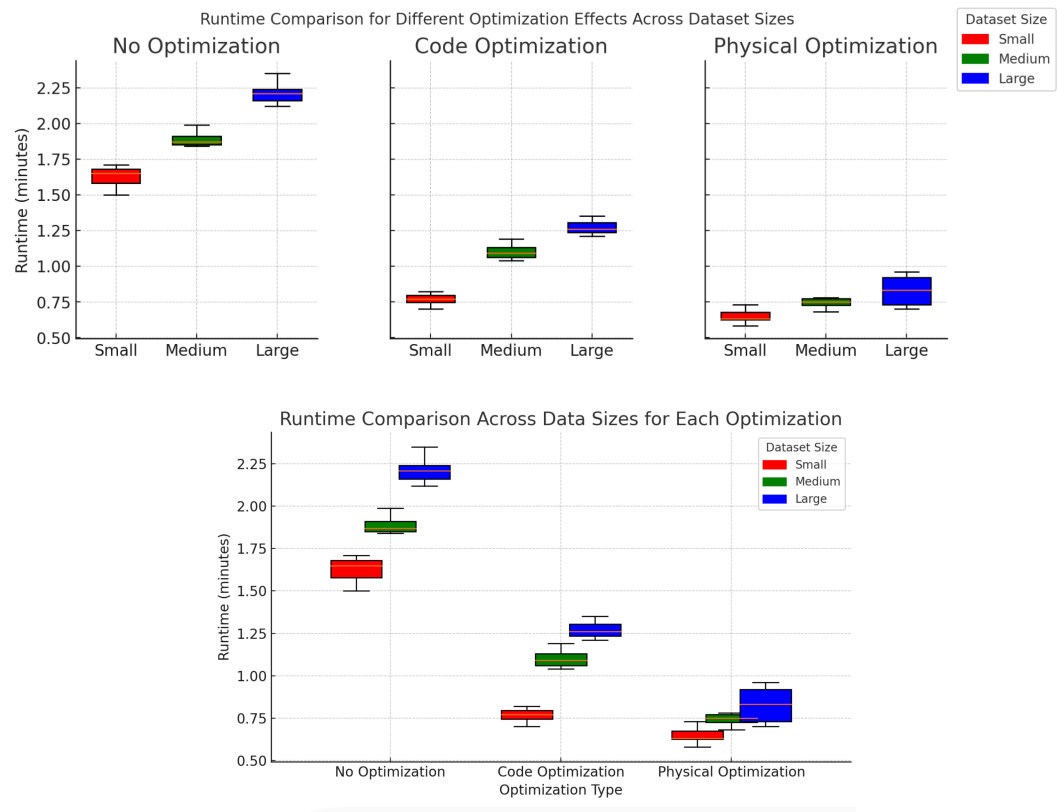
This is particularly beneficial for the reviews table, which contains a large amount of irrelevant data. By organizing the Parquet files with a directory structure, we only load the subdirectory for the year 2024, completely avoiding unnecessary data scanning—a major advantage of the Parquet format. In addition to changes at the storage level, all prior code-level optimizations are retained to maximize efficiency.

Result Comparison and Visualization

No Optimise				Code Optimise				Physical Optimise			
Measure#	Small	Medium	Large	Measure#	Small	Medium	Large	Measure#	Small	Medium	Large
1	1.53	1.85	2.12	1	0.79	1.06	1.33	1	0.73	0.75	0.7
2	1.68	1.87	2.21	2	0.77	1.15	1.24	2	0.62	0.7	0.94
3	1.71	1.99	2.35	3	0.73	1.11	1.26	3	0.63	0.75	0.9
4	1.68	1.87	2.24	4	0.75	1.07	1.3	4	0.69	0.78	0.82
5	1.5	1.89	2.24	5	0.77	1.06	1.21	5	0.63	0.85	0.96
6	1.65	1.93	2.02	6	0.81	1.19	1.22	6	0.66	0.68	0.76
7	1.67	1.84	2.15	7	0.74	1.19	1.25	7	0.58	0.76	0.7
8	1.65	1.85	2.28	8	0.82	1.04	1.31	8	0.63	0.7	0.95
9	1.51	1.85	2.17	9	0.7	1.04	1.23	9	0.72	0.89	0.7
10	1.71	1.94	2.23	10	0.8	1.09	1.35	10	0.61	0.75	0.88
Mean	1.629	1.89	2.20	Mean	0.768	1.10	1.27	Mean	0.65	0.761	0.831

Execution Time Comparison Across Small, Medium, and Large Datasets (min) No Optimization/Code Optimization/Physical Optimization

From the table above, we can clearly observe the execution times under three different scenarios: without optimization (left), with code optimization (middle), and with physical optimization (right), across small, medium, and large datasets. For each scenario, the query was executed 10 times, and the times were recorded in detail. The unit of measurement is minutes (min). It is evident that as the dataset size increases, the execution time also rises steadily, which is closely related to the increasing computational workload. At the same time, with each level of optimization, we see varying degrees of performance improvement. This comparison becomes even more intuitive when viewed through the Boxplot visualization, which clearly illustrates the impact of each optimization approach.



The figure above compares the execution times across three optimization strategies—No Optimization, Code Optimization, and Physical Optimization—for three dataset sizes: Small (red), Medium (green), and Large (blue). The chart clearly shows that the applied optimizations significantly improve performance, especially for medium and large datasets. Physical optimization demonstrates the most substantial performance gain, achieving the shortest execution times. In contrast, the unoptimized queries consistently show the longest execution times across all dataset sizes. This highlights the critical importance of applying proper optimization techniques—particularly at the physical layer—when working with larger-scale data, as it greatly enhances processing efficiency.

Execution Plan

The specific optimization effects can be observed directly from the Execution Plan. The key improvements are reflected as follows:

```

Batched: false, DataFilters: [isNotNull(city_id#127), isNotNull(neighbourhood#128)], Format: CSV, Location: InMemoryFileIndex(1
paths)[dbfs:/FileStore/tables/airbnb_listings-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(city_id), IsNotNull(neighbourhood)], ReadSchema:
struct<id:bigint,listing_name:string,city_id:double,neighbourhood:double>

[id#2149,city_id#2156,neighbourhood#2157] Batched: true, DataFilters: [isNotNull(city_id#2156), isNotNull(neighbourhood#2157)], Format: Parquet, Location:
InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_listings-large.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(city_id),
IsNotNull(neighbourhood)], ReadSchema: struct<id:string,city_id:string,neighbourhood:string>

AND (year(review_date#1942) = 2024)

+- *(1) Filter (isNotNull(review_date#1942))
+- *(1) ColumnarToRow
+- FileScan parquet
[listing_id#1941,review_date#1942] Batched: true, DataFilters: [isNotNull(review_date#1942), (year(review_date#1942) = 2024)], Format: Parquet, Location:
InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_reviews_by_year-large.parquet/yr..., PartitionFilters: [], PushedFilters:
[IsNotNull(review_date)], ReadSchema: struct<listing_id:string,review_date:date>

```

In the unoptimized version, all input data is read in CSV format using FileScan csv, and then converted to row format via ColumnarToRow for further computation. In contrast, the physical optimization replaces the main data source with Parquet format, using FileScan parquet for direct columnar scanning. This enables Spark to take advantage of column pruning and predicate pushdown.

This optimization is especially impactful for the reviews table, which contains a large volume of data. We applied partition pruning specifically for this dataset, leveraging the year-based directory structure to read only the necessary partitions (e.g., for year = 2024). By avoiding irrelevant columns entirely and applying early filtering (e.g., year = 2024, city = 'Sydney') along with early projection, we significantly reduced I/O operations and serialization overhead.

```

+- *(8) SortMergeJoin [listing_id#1941], [listing_id#2017], Inner
:- Sort [listing_id#1941 ASC NULLS FIRST], false, 0
: +- Exchange hashpartitioning(listing_id#1941, 200), ENSURE_REQUIREMENTS,
+- ResolvedHint (strategy=broadcast)
+- Project [id#1552, nhood_name#1554, city_id#1553]
+- Relation

```

```

+- *(20) BroadcastHashJoin [neighbourhood_name#1352], [neighbourhood_name#1557], Inner, BuildRight, false, false
:- *(20) Project [listing_name#1207, neighbourhood_name#1352, least((coalesce(nanvl(occupation_days_sum#1413,
null), 0.0) / 365.0), 1.0) AS occupancy_rate#1472]

: +- *(3) BroadcastHashJoin

[knownfloatingpointnormalized(normalizenanandzero(city_id#127))], [knownfloatingpointnormalized(normalizenanandzero(cast(id#93 as double)))], Inner,
BuildRight, false, false

```

In the unoptimized query, when two tables are joined, the execution engine defaults to using ShuffleHashJoin or SortMergeJoin, without explicitly applying broadcast strategies. In contrast, the optimized execution plan introduces multiple instances of ResolvedHint (strategy = broadcast), enabling broadcast joins for small tables such as the cities and neighbourhoods tables. This significantly reduces shuffle overhead. These broadcast join operations are clearly marked in the plan, indicating that Spark is using the small tables efficiently by distributing them to all executors for local joining.

```

① *(21) RunningWindowFunction [...] row_number() windowdefinition(neighbourhood_name#342, occupancy_rate#667 DESC NULLS LAST) AS rn#727],
[neighbourhood_name#342], [occupancy_rate#667 DESC NULLS LAST], false
+- *(21) Sort [neighbourhood_name#342 ASC NULLS FIRST, occupancy_rate#667 DESC NULLS LAST]
...
Filter (rn#727 <= 5)
.....
*(32) HashAggregate(keys=[neighbourhood_name#823], functions=[finalmerge_avg(merge_sum#954, count#955L) AS avg(occupancy_rate#667)#722],
output=[neighbourhood_name#823, average_occupancy_rate#723])
+- *(31) HashAggregate(keys=[neighbourhood_name#823], functions=[partial_avg(occupancy_rate#667) AS (sum#954, count#955L)], output=[...])

② +- Filter (rn#1577 <= 5)
+- Window [...] row_number() windowdefinition(neighbourhood_name#1352, occupancy_rate#1472 DESC NULLS LAST) AS rn#1577],
[neighbourhood_name#1352], [occupancy_rate#1472 DESC NULLS LAST]
+- *(21) Sort [neighbourhood_name#1352 ASC NULLS FIRST, occupancy_rate#1472 DESC NULLS LAST]
.....
*(22) Aggregate [neighbourhood_name#1352, average_occupancy_rate#1492],
[neighbourhood_name#1352, average_occupancy_rate#1492, collect_list(...rn#1577...) AS listings_struct#1601]
+- *(21) Project [neighbourhood_name#1352, knownfloatingpointnormalized(normalizenanandzero(average_occupancy_rate#1492)) AS
average_occupancy_rate#1492, rn#1577, listing_name#1207, occupancy_rate#1472]
+- *(21) Filter (rn#1577 <= 5)

```

The execution plans above illustrate how multiple computational steps were merged into a more efficient unified operation after optimization.

In the first execution plan (1), a RunningWindowFunction is used along with a Filter (rn <= 5) to compute the top 5 listings separately. Then, a separate HashAggregate stage is triggered to calculate the average occupancy rate. This process involves two physical operators, two stages, and two shuffle operations, which increases computation time and resource usage.

In contrast, the optimized execution plan (2) combines these steps in a single branch. The Window + Filter (rn <= 5) is followed immediately by the Aggregate operation. This approach requires only one shuffle and one stage, significantly reducing the computational overhead and improving execution efficiency. You can check the Execution plan at Appendix C: Task3 Execution plan(Page 16-23).

Stage Matrix

Finally, we conducted a comparative analysis using the Stage Matrix to evaluate performance across the small, medium, and large datasets under different levels of optimization. The table presents

detailed execution metrics, including Elapsed Time, which represents the actual time taken from the start to the end of a stage; Executor Run Time, indicating the total time spent by tasks actively executing; and GC Time, reflecting the time spent on garbage collection, which may signal inefficient memory usage, data skew, or excessive caching if the value is high. Additionally, Shuffle Write and Shuffle Read show the amount of data exchanged during shuffle operations, while Task Count and Stage Count represent the total number of tasks and the number of execution stages in the query, respectively. Each stage comprises a set of tasks that run in parallel. These metrics provide a comprehensive view of how each optimization strategy impacts query efficiency and resource utilization.

Metric	No Optimization	Code Optimized	Physical Optimized
Elapsed Time (min)	1.63	0.75 (-54%)	0.67 (-59%)
Executor Run Time (min)	2.21	1.57 (-29%)	0.89 (-60%)
GC Time (s)	4.73	2.94 (-38%)	2.06 (-56%)
Shuffle Write / Read (MB)	2.02 / 2.66	1.36 / 1.42	0.60 / 0.60
Task Count	2240	847 (-62%)	827 (-63%)
Stage Count	29	22 (-24%)	19 (-34%)

Spark Stage Performance Comparison Across Optimization Strategies (Small dataset)

Metric	No Optimization	Code Optimized	Physical Optimized
Elapsed Time (min)	1.89	1.04 (-45%)	0.73 (-61%)
Executor Run Time (min)	4.82	3.99 (-17%)	1.38 (-71%)
GC Time (s)	11.61	9.44 (-19%)	3.73 (-68%)
Shuffle Write / Read (MB)	8.8 / 11.6	6.1 / 6.3	10.1 / 10.4
Task Count	2260	867 (-62%)	839 (-63%)
Stage Count	29	22 (-24%)	18 (-38%)

Spark Stage Performance Comparison Across Optimization Strategies (Medium dataset)

Metric	No Optimization	Code Optimized	Physical Optimized
Elapsed Time (min)	2.20	1.31 (-40%)	0.79 (-64%)
Executor Run Time (min)	6.85	6.05 (-12%)	1.79 (-74%)
GC Time (s)	11.25	8.79 (-22%)	5.36 (-52%)
Shuffle Write / Read (MB)	16.5 / 22.0	12.1 / 12.4	21.2 / 21.7
Task Count	2260	867 (-62%)	839 (-63%)
Stage Count	29	22 (-24%)	18 (-38%)

Spark Stage Performance Comparison Across Optimization Strategies (Large dataset)

Using the large dataset as an example, we observed a stepwise improvement in query performance as optimization techniques were progressively applied. In the unoptimized state, the query took 2.2 minutes (Elapsed Time) to complete, with an Executor Run Time of 6.85 minutes and a GC Time of 11 seconds. This indicates significant execution time, a high number of tasks, and considerable

memory overhead due to intermediate data shuffling and I/O operations. After applying code-level optimizations—such as early filtering and broadcast joins for small tables—the Elapsed Time dropped to 1.31 minutes, Executor Run Time was reduced to 6.05 minutes, and GC Time decreased by 22%. With further physical optimizations—including switching from CSV to Parquet, enabling column pruning, vectorized reading, and refining join strategies—the total query time dropped to 0.79 minutes, and Executor Run Time was cut down to 1.49 minutes, representing a more than 52% reduction from the baseline. GC Time was further reduced to 5 seconds. Additionally, the number of stages decreased from 29 to 18, and tasks from 2260 to 839, indicating better operator fusion and execution efficiency. While Shuffle Read/Write increased to 21/22 MB due to finer partitioning, the overall performance gains from physical optimization were substantial. Code optimization not only improved performance but also laid a solid foundation for the effectiveness of physical-level enhancements. Similar trends were observed across small and medium datasets, where both optimization strategies demonstrated consistent and significant improvements in performance.

Overall, for task3, we still choose physical optimization as the preferred solution. Its advantages in improving performance can already be well demonstrated, greatly helping us save waiting time.

Group contribution statement:

████████	Member A	Major contributor (yes)
████████	Yuanfeng Liu	Major contributor (yes)

Appendix A: Task3 Result

Task3 Result-Small dataset

|Rockdale\t0.27\t[(Lucky Lounge Room, 1.0), (Cozy Lounge Room, 1.0), (1br near airport w/ private access and parking, 1.0), (Heart of Rockdale 3, 1.0), (Modern 2 Bed Apartment. Close to city and airports, 0.8383561643835616)]
|
|Sydney\t0.23\t[(Entire studio apartment, 1.0), (Canopy Room in Zen Home near Botanic Gardens. Rm3, 1.0), (KozyGuru | Sydney Cbd 2 Bed Apt | City View, 1.0), (Admire Harbour Views from a Hip Potts Point Sanctuary, 1.0), (Superb 4-Br Apt Darling Harbour View: Heart of CBD, 1.0)]
|
|Waverley\t0.22\t[(Splash - 2 bdr opposite bondi beach, 1.0), (The Arc Bondi Beach, 1.0), (Bondi beachfront pad - amazing views of the beach!, 1.0), (Your home by the ocean., 1.0), (200m to the beach ~ Ocean view in every direction, 1.0)]
|
|Blacktown\t0.21\t[(Spacious 2-BR Home in Seven Hills - Free Parking, 1.0), (Swimming Pool - Enjoy summer and relax, 0.7890410958904109), (Golf-Park-View Spacious 6-Bedroom + Free Parking, 0.7232876712328767), (1 Room-1 Single bed - 4 min walk to Toongabbie Stn, 0.5095890410958904), (R Double bed 1-2, fan heater, laptop table, 0.4602739726027397)]
|
|Fairfield\t0.21\t[(Spacious 1-bedroom guest house with self check in!, 0.8712328767123287), (Single room 2 bed suit 1-3 Fairfield 2165 - Sydney, 0.4767123287671233), (Garden studio, 0.4602739726027397), (Convenient New Private granny flat, 0.4438356164383562), (Resort style one bedroom APT near Glof club, 0.2958904109589041)]
|
|Hornsby\t0.20\t[(Cherrybrook Studio Apartment, 0.8876712328767123), (Epping Garden 2, 0.5753424657534246), (Luxury Moderne in Tranquil Cliff Top Bush Retreat, 0.536986301369863), (The True Blue House with SPA & Pool, 0.4602739726027397), (Clean & comfy 2-BR Granny Flat close to Bus/Train, 0.4383561643835616)]
|
|North Sydney\t0.19\t[(Harbour Bridge King Room, 1.0), (Private Double Kitchenette Room - Shared Bathroom, 1.0), (Superb Apt in an Great Location !, 1.0), (Cosy Stay @ North Sydney, 0.7890410958904109), (Spacious Apartment at the Best Location, 0.7232876712328767)]

|
|Randwick\t0.19\t[(A Beautiful Ocean Front Apartment in Clovelly, 1.0), (Coogee Beachside Retreat, 1.0), (Apartment directly on beach with stunning views, 0.9698630136986301), (Jumanji-private room 5 min UNSW, 0.9205479452054794), (Stylish Studio in Maroubra, 0.9041095890410958)]
|
|Burwood\t0.19\t[(New stylish townhouse in Burwood, 0.5589041095890411), (New beautiful townhouse in Burwood, 0.5095890410958904), (Studio Apartment in Strathfield CBD B5, 0.4931506849315068), (Sydney Burwood 4 bedroom Guest house, 0.410958904109589), (Cozy Croydon Park home with leafy green garden, 0.39452054794520547)]
|
|Marrickville\t0.18\t[(Double bed very close to airport, 0.936986301369863), (Stylish Hipster Heaven in Camperdown, 0.8493150684931506), (* Enmore's Little Hidden Gem *, 0.5753424657534246), (A Private Room with Two single beds, 0.5424657534246575), (King's Hideaway Studio, 0.5095890410958904)] |

Task3 Result - Medium dataset

|Leichhardt\t0.25\t[(JS8 - Spacious Budget Studio Near CBD, 1.0), (JS5 - Affordable Easy Stay w/ Kitchenette, 1.0), (JS6 - Spacious Affordable Room 15mins bus to City, 1.0), (JS6A - Nice Budget Studio*Bus to City, 1.0), (JS4 - Budget Sydney City Escape w/ Kitchenette, 1.0)]
|
|Hunters Hill\t0.25\t[(unit to yourself Hunters Hill, 1.0), (Artist's studio loft, 0.3287671232876712), (Large Queen Studio with Private Entrance, 0.2958904109589041), (Gladesview- Luxury 2br with views, secure parking, 0.13150684931506848), (Prime waterfront Bay Window Room, 0.09863013698630137)]
|
|Sydney\t0.23\t[(Bed in 4-Bed Mixed Dorm with En-suite (Age 18-40), 1.0), (Hotel-like private suite with bathroom & city view, 1.0), (Darling Harbour Wharfs ICC City long stay 20%off, 1.0), (Waterfront - Designer Curated @ The Rocks Sydney, 1.0), (New York Style Three-story Loft near CBD, 1.0)]
|
|Rockdale\t0.21\t[(Close to the City, Airport, Train Station & Beach, 1.0), (Aircon Studio w/ private bathroom & kitchen (Rm 0), 1.0), (Cozy Lounge Room, 1.0), (Brighton Le sands 2 BDR Apartment Stylish, 1.0), (Heart of Rockdale 3, 1.0)]
|
|Ashfield\t0.21\t[(1 large light filled bedroom available to rent., 1.0), (Garden Studio in Ashfield, 1.0), (Stunning apartment in historic mansion, 0.9698630136986301), (large creative home & garden close to CBD., 0.8767123287671232), (Lovely studio in Summer Hill/A06, 0.8767123287671232)]
|
|Marrickville\t0.21\t[(5 min walk to train, shops and entertainment, 1.0), (Spacious inner west flat with roof top garden, 1.0), (Newtown Escape, 1.0), (Cosy Studio in Best Location - 1min to shops!, 1.0), (St Peters 1 bed flat-11am ch/out. No cleaning fee., 1.0)]
|
|North Sydney\t0.20\t[(4.Harbour View Studio: Sydney's Scenic Hideaway, 1.0), (Kirribilli 2 BDR minutes from view of Opera House, 1.0), (Harbour Bridge King Room, 1.0), (Cosy 2 Bed Apt @ St Leonards & Stunning Views, 1.0), (Cozy nest

in Crows Nest, 1.0]]
|
|Hurstville\t0.19\t[(Cozy & Bright 1 bed Apt in Heart of Hurstville, 1.0), (The Goblet Tiny Home, 0.9698630136986301), (Cheerful 3bed home FreeP/Close to the station, 0.7890410958904109), (001.hurstville 舒适温馨的小公寓, 1 分钟到火车站。周围超市, 大型商场, 电影, 美食街, 0.7068493150684931), (Upstairs Retreat. Stylish, comfortable+ breakfast, 0.6246575342465753)]
|
|Fairfield\t0.18\t[(Entire (no sharing) 2 bedroom granny flat@ thepark, 0.936986301369863), (Super Convenience 2 Great br 650mt to train, 0.7232876712328767), (Comfortable new 2-bedroom with parking, 0.7232876712328767), (Canley Vale 2 Large Bedroom unit, 0.7068493150684931), (Private sleep out ensuite with separate entrance, 0.6904109589041096)]
|
|Waverley\t0.18\t[(Cozy hotel studio near Bondi Junction station H01, 1.0), (Bondi Beach quiet renovated 1 bed apt, 1.0), (Modern oasis in Bondi, 1.0), (Rustic home by the sea -Bronte beach view, 1.0), (Stylish Bondi Pad - 3 mins to Surf & Sand, 1.0)] |

Task3 Result- Large dataset

|Sydney\t0.23\t[(New York inspired city pad in Potts Point, 1.0), (Bed in 4-Bed Mixed Dorm with En-suite (Age 18-40), 1.0), (Hotel-like private suite with bathroom & city view, 1.0), (Entire studio apartment, 1.0), (Darling Harbour Wharfs ICC City long stay 20%off, 1.0)]
|
|Leichhardt\t0.22\t[(JS8 - Spacious Budget Studio Near CBD, 1.0), (JS5 - Affordable Easy Stay w/ Kitchenette, 1.0), (JS6 - Spacious Affordable Room 15mins bus to City, 1.0), (JS6A - Nice Budget Studio*Bus to City, 1.0), (JS4 - Budget Sydney City Escape w/ Kitchenette, 1.0)]
|
|North Sydney\t0.22\t[(4.Harbour View Studio: Sydney's Scenic Hideaway, 1.0), (Comfy 2 Bed Apt & Stunning Views @ St Leonards, 1.0), (Kirribilli 2 B DR minutes from view of Opera House, 1.0), (Harbour Bridge King Room, 1.0), (Private Single Kitchenette Room - Shared Bathroom, 1.0)]
|
|Marrickville\t0.21\t[(Contemporary Camperdown Studio, 1.0), (5 min walk to train, shops and entertainment, 1.0), (Newtown's-Enmore Theater, performing arts precinct, 1.0), (Camperdown Core - apt with aircon & parking, 1.0), (Spacious inner west flat with roof top garden, 1.0)]
|
|Camden\t0.20\t[(Self contained flat at rear of Lake side residence, 1.0), (Cozy room with separate bathroom., 1.0), (Private Rural Retreat with bus h Views!, 0.9698630136986301), (Guest house in Harrington Park, 0.7890410958904109), (Entire 2 Bedroom Brand new Home with free parking., 0.7397260273972602)]
|
|Rockdale\t0.19\t[(LUXURY VILLA ON THE CITY FRINGE, 1.0), (Brighton-le-Sands, private bedroom & bathroom, 1.0), (Close to the City, Airport, Train Station & Beach, 1.0), (Aircon Studio w/ private bathroom & kitchen (Rm 0), 1.0), (Lucky Lounge Room, 1.0)]
|
|Hunters Hill\t0.18\t[(unit to yourself Hunters Hill, 1.0), (Spacious Family Home in Sydney's northern suburbs., 0.9205479452054794), (Cozy and Modern 3-Bedroom Home, 0.3780821917808219), (Artist's studio loft, 0.3287671232876712), (Large Queen Studio with Private Entrance, 0.2958904109589041)] ,
|
|Burwood\t0.18\t[(City View 2BR nr Plaza and train station+2FPKG, 1.0), (2 Bedroom Apt opposite westfield + Free Parking, 1.0), (Lovely studio apartment in Strathfield CBD H2, 0.9863013698630136), (A Lovely Home - 12 mins train to the city, 0.958904109589041), (City-View 2BR nr Plaza & train station + FPK, 0.8876712328767123)]
|
|Sutherland Shire\t0.18\t[(Bundeena Treehouse with Outdoor Spa and Views, 1.0), (Waterfront on Botany Bay., 1.0), (Comfy and close to transport, 1.0), (Sydney Meets Miranda, Entire 4 Bedroom home,7 beds, 1.0), (Panoramic Ocean and Royal National Park views, 1.0)]
|
|Waverley\t0.18\t[(Stylish City and Beach Apartment, 1.0), (Cozy hotel studio near Bondi Junction station H01, 1.0), (Bondi Beach quiet renovated 1 bed apt, 1.0), (Modern oasis in Bondi, 1.0), (Bondi Beach Retreat - light & airy studio in Bondi, 1.0)]
|

Appendix B: Task2 Execution Plan

Without Optimization

```
== Physical Plan ==
TakeOrderedAndProject(limit=5, orderBy=[num_reviews#1406L DESC NULLS LAST,listing_name#1077 ASC NULLS FIRST], output=[listing_name#1077,city_name#1159,num_reviews#1406L])
+- *(7) HashAggregate(keys=[listing_name#1077, city_name#1159], functions=[finalmerge_count(distinct merge count#1421L) AS count(id#1129L)#1407L], output=[listing_name#1077, city_name#1159, num_reviews#1406L])
  +- Exchange hashpartitioning(listing_name#1077, city_name#1159, 200), ENSURE_REQUIREMENTS, [plan_id=3076]
    +- *(6) HashAggregate(keys=[listing_name#1077, city_name#1159], functions=[partial_count(distinct id#1129L) AS count#1421L], output=[listing_name#1077, city_name#1159, count#1421L])
      +- *(6) HashAggregate(keys=[listing_name#1077, city_name#1159, id#1129L], functions=[], output=[listing_name#1077, city_name#1159, id#1129L])
        +- Exchange hashpartitioning(listing_name#1077, city_name#1159, id#1129L, 200), ENSURE_REQUIREMENTS, [plan_id=3071]
          +- *(5) HashAggregate(keys=[listing_name#1077, city_name#1159, id#1129L], functions=[], output=[listing_name#1077, city_name#1159, id#1129L])
            +- *(5) Project [listing_name#1077, id#1129L, city_name#1159]
              +- *(5) BroadcastHashJoin [knownfloatingpointnormalized(normalizenanandzero(city_id#1083))], [knownfloatingpointnormalized(normalizenanandzero(cast(id#1158 as double)))], Inner, BuildRight, false, false
                :- *(5) Project [listing_name#1077, city_id#1083, id#1129L]
                  : +- *(5) SortMergeJoin [id#1076L], [listing_id#1130L], Inner
                  :   :- Sort [id#1076L ASC NULLS FIRST], false, 0
                  :     : +- Exchange hashpartitioning(id#1076L, 200), ENSURE_REQUIREMENTS, [plan_id=3049]
                  :       : +- *(2) Project [id#1076L, listing_name#1077, city_id#1083]
                  :         : +- *(2) BroadcastHashJoin [id#1041], [cast(host_id#1082 as int)], Inner, BuildLeft, false, false
                  :           : :- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=3041]
                  :             : +- *(1) Project [id#1041]
                  :               : +- *(1) Filter ((isnotnull(is_superhost#1045) AND (is_superhost#1045 = t)) AND isnotnull(id#1041))
                  :                 : +- FileScan csv [id#1041,is_superhost#1045] Batched: false, DataFilters: [isnotnull(is_superhost#1045), (is_superhost#1045 = t), isnotnull(id#1041)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_hosts.csv], PartitionFilters: [], PushedFilters: [IsNotNull(is_superhost), EqualTo(is_superhost,t), IsNotNull(id)], ReadSchema: struct<id:int,is_superhost:string>
                  :                   : +- *(2) Project [id#1076L, listing_name#1077, host_id#1082, city_id#1083]
                  :                     : +- *(2) Filter (((isnotnull(room_type#1079) AND (room_type#1079 = Entire home/apt)) AND isnotnull(host_id#1082)) AND isnotnull(id#1076L)) AND isnotnull(city_id#1083))
                  :                       : +- FileScan csv [id#1076L,listing_name#1077,room_type#1079,host_id#1082,city_id#1083] Batched: false, DataFilters: [isnotnull(room_type#1079), (room_type#1079 = Entire home/apt), isnotnull(host_id#1082), isnotnull(id#1076L)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_listings-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(room_type), EqualTo(room_type,Entire home/apt), IsNotNull(host_id), IsNotNull(id), IsNotNull(city_id)], ReadSchema: struct<id:bigint,listing_name:string,room_type:string,host_id:string,city_id:double>
                  :                         : +- Sort [listing_id#1130L ASC NULLS FIRST], false, 0
                  :                           : +- Exchange hashpartitioning(listing_id#1130L, 200), ENSURE_REQUIREMENTS, [plan_id=3056]
                  :                             : +- *(3) Project [id#1129L, listing_id#1130L]
                  :                               : +- *(3) Filter ((isnotnull(review_date#1131) AND (year(review_date#1131) = 2025)) AND isnotnull(listing_id#1130L))
                  :                                 : +- FileScan csv [id#1129L,listing_id#1130L,review_date#1131] Batched: false, DataFilters: [isnotnull(review_date#1131), (year(review_date#1131) = 2025), isnotnull(listing_id#1130L)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_reviews-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(review_date), IsNotNull(listing_id)], ReadSchema: struct<id:bigint,listing_id:bigint,review_date:date>
                  :                                   +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(knownfloatingpointnormalized(normalizenanandzero(cast(input[0, int, true] as double))))), [plan_id=3065]
                  :                                     +- *(4) Project [id#1158, city_name#1159]
                  :                                       +- *(4) Filter ((isnotnull(country_code#1162) AND (country_code#1162 = AU)) AND isnotnull(id#1158))
                  :                                         +- FileScan csv [id#1158,city_name#1159,country_code#1162] Batched: false, DataFilters: [isnotnull(country_code#1162), (country_code#1162 = AU), isnotnull(id#1158)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_cities.csv], PartitionFilters: [], PushedFilters: [IsNotNull(country_code), EqualTo(country_code,AU), IsNotNull(id)], ReadSchema: struct<id:int,city_name:string,country_code:string>
```

Code optimization

```
== Physical Plan ==
TakeOrderedAndProject(limit=5, orderBy=[num_reviews#9519L DESC NULLS LAST,listing_name#9071 ASC NULLS FIRST], output=[listing_name#9071,city_name#9153,num_reviews#9519L])
+- *(6) HashAggregate(keys=[listing_name#9071, city_name#9153], functions=[finalmerge_count(distinct merge count#9554L) AS count(id#9123L)#9520L], output=[listing_name#9071, city_name#9153, num_reviews#9519L])
  +- Exchange hashpartitioning(listing_name#9071, city_name#9153, 200), ENSURE_REQUIREMENTS, [plan_id=31919]
    +- *(5) HashAggregate(keys=[listing_name#9071, city_name#9153], functions=[partial_count(distinct id#9123L) AS count#9554L], output=[listing_name#9071, city_name#9153, count#9554L])
      +- *(5) HashAggregate(keys=[listing_name#9071, city_name#9153, id#9123L], functions=[], output=[listing_name#9071, city_name#9153, id#9123L])
        +- Exchange hashpartitioning(listing_name#9071, city_name#9153, id#9123L, 200), ENSURE_REQUIREMENTS, [plan_id=31914]
          +- *(4) HashAggregate(keys=[listing_name#9071, city_name#9153, id#9123L], functions=[], output=[listing_name#9071, city_name#9153, id#9123L])
            +- *(4) Project [listing_name#9071, id#9123L, city_name#9153]
              +- *(4) BroadcastHashJoin [knownfloatingpointnormalized(normalizenanandzero(city_id#9077))], [knownfloatingpointnormalized(normalizenanandzero(cast(id#9152 as double)))], Inner, BuildRight, false, false
                :- *(4) Project [listing_name#9071, city_id#9077, id#9123L]
                  : +- *(4) BroadcastHashJoin [id#9070L], [listing_id#9124L], Inner, BuildRight, false, false
                  :   :- *(4) Project [id#9070L, listing_name#9071, city_id#9077]
                  :     : +- *(4) BroadcastHashJoin [id#9035], [cast(host_id#9076 as int)], Inner, BuildLeft, false, false
                  :       : :- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=3189]
```

```

0]
      :      :      :      +- *(1) Project [id#9035]
      :      :      :      +- *(1) Filter (((isnotnull(is_superhost#9039) AND (is_superhost#9039 = t)) AND isnotnull(id#9035))
      :      :      :      +- FileScan csv [id#9035,is_superhost#9039] Batched: false, DataFilters: [isnotnull(is_superhost#9039),
(is_superhost#9039 = t), isnotnull(id#9035)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_hosts.csv], PartitionF
ilters: [], PushedFilters: [IsNotNull(is_superhost), EqualTo(is_superhost,t), IsNotNull(id)], ReadSchema: struct<id:int,is_superhost:string>
      :      :      +- *(4) Project [id#9070L, listing_name#9071, host_id#9076, city_id#9077]
      :      :      +- *(4) Filter (((isnotnull(room_type#9073) AND (room_type#9073 = Entire home/apt)) AND isnotnull(host_id#90
76)) AND isnotnull(id#9070L)) AND isnotnull(city_id#9077))
      :      :      +- FileScan csv [id#9070L,listing_name#9071,room_type#9073,host_id#9076,city_id#9077] Batched: false, Data
Filters: [isnotnull(room_type#9073), (room_type#9073 = Entire home/apt), isnotnull(host_id#9076), isnotnul..., Format: CSV, Location: InMemoryFileIn
dex(1 paths)[dbfs:/FileStore/tables/airbnb_listings-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(room_type), EqualTo(room_type,Entire
home/apt), IsNotNull(host_id), IsNotNull(id), IsN..., ReadSchema: struct<id:bigint,listing_name:string,room_type:string,host_id:string,city_id:doubl
e>
      :      +- BroadcastExchange HashedRelationBroadcastMode(List(input[1, bigint, false]),false), [plan_id=31900]
      :      +- *(2) Filter isnotnull(listing_id#9124L)
      :      +- InMemoryTableScan [id#9123L, listing_id#9124L], [isnotnull(listing_id#9124L)], false
      :      +- InMemoryRelation [id#9123L, listing_id#9124L], StorageLevel(disk, memory, serialized, 1 replicas)
      :      +- *(1) Project [id#9123L, listing_id#9124L]
      :      +- *(1) Filter (isnotnull(review_date#9125) AND (year(review_date#9125) = 2025))
      :      +- FileScan csv [id#9123L,listing_id#9124L,review_date#9125] Batched: false, DataFilters: [isn
otnull(review_date#9125), (year(review_date#9125) = 2025)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_reviews-
large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(review_date)], ReadSchema: struct<id:bigint,listing_id:bigint,review_date:date>
      +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(knownfloatingpointnormalized(normalizenanandzero(cast(input[0,
int, true] as double))))),false), [plan_id=31908]
      +- *(3) Project [id#9152, city_name#9153]
      +- *(3) Filter ((isnotnull(country_code#9156) AND (country_code#9156 = AU)) AND isnotnull(id#9152))
      +- FileScan csv [id#9152,city_name#9153,country_code#9156] Batched: false, DataFilters: [isnotnull(country_code#915
6), (country_code#9156 = AU), isnotnull(id#9152)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_cities.csv], Part
itionFilters: [], PushedFilters: [IsNotNull(country_code), EqualTo(country_code,AU), IsNotNull(id)], ReadSchema: struct<id:int,city_name:string,coun
try_code:string>

```

Physical optimization

```

== Physical Plan ==
TakeOrderedAndProject(limit=5, orderBy=[num_reviews#9832L DESC NULLS LAST,listing_name#9721 ASC NULLS FIRST], output=[listing_name#9721,city_name#97
11,num_reviews#9832L])
+- *(6) HashAggregate(keys=[listing_name#9721, city_name#9711], functions=[finalmerge_count(distinct merge count#9847L) AS count(id#9756)#9833L], ou
tput=[listing_name#9721, city_name#9711, num_reviews#9832L])
  +- Exchange hashpartitioning(listing_name#9721, city_name#9711, 200), ENSURE_REQUIREMENTS, [plan_id=32828]
    +- *(5) HashAggregate(keys=[listing_name#9721, city_name#9711], functions=[partial_count(distinct id#9756) AS count#9847L], output=[listing_na
me#9721, city_name#9711, count#9847L])
      +- *(5) HashAggregate(keys=[listing_name#9721, city_name#9711, id#9756], functions=[], output=[listing_name#9721, city_name#9711, id#9756])
      +- Exchange hashpartitioning(listing_name#9721, city_name#9711, id#9756, 200), ENSURE_REQUIREMENTS, [plan_id=32823]
        +- *(4) HashAggregate(keys=[listing_name#9721, city_name#9711, id#9756], functions=[], output=[listing_name#9721, city_name#9711, id#
9756])
          +- *(4) Project [listing_name#9721, id#9756, city_name#9711]
            +- *(4) BroadcastHashJoin [city_id#9727], [id#9710], Inner, BuildRight, false, false
              :- *(4) Project [listing_name#9721, city_id#9727, id#9756]
                +- *(4) BroadcastHashJoin [id#9720], [listing_id#9757], Inner, BuildRight, false, false
                  :- *(4) Project [id#9720, listing_name#9721, city_id#9727]
                    : +- *(4) BroadcastHashJoin [id#9692], [host_id#9726], Inner, BuildLeft, false, false
                      : :- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string, true]),false), [plan_id=32796]
                        : +- *(1) Project [id#9692]
                          :
                          : +- *(1) Filter (((isnotnull(is_superhost#9696) AND (is_superhost#9696 = t)) AND isnotnull(id#9692))
                          : +- *(1) ColumnarToRow
                          : +- FileScan parquet [id#9692,is_superhost#9696] Batched: true, DataFilters: [isnotnull(is_superhost#
9696), (is_superhost#9696 = t), isnotnull(id#9692)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_hos
ts.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(is_superhost), EqualTo(is_superhost,t), IsNotNull(id)], ReadSchema: struct<id:string,is
_superhost:string>
                          : +- *(4) Project [id#9720, listing_name#9721, host_id#9726, city_id#9727]
                          : +- *(4) Filter (((isnotnull(room_type#9723) AND (room_type#9723 = Entire home/apt)) AND isnotnull(host_id#97
26)) AND isnotnull(id#9720)) AND isnotnull(city_id#9727))
                          : +- *(4) ColumnarToRow
                          : +- FileScan parquet [id#9720,listing_name#9721,room_type#9723,host_id#9726,city_id#9727] Batched: true,
DataFilters: [isnotnull(room_type#9723), (room_type#9723 = Entire home/apt), isnotnull(host_id#9726), isnotnul..., Format: Parquet, Location: InMemo
ryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_listings-large.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(room_type), Equa
lTo(room_type,Entire home/apt), IsNotNull(host_id), IsNotNull(id), IsN..., ReadSchema: struct<id:string,listing_name:string,room_type:string,host_i
d:string,city_id:string>
                          : +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[1, string, false]),false), [plan_id=32808]
                          : +- *(2) Filter isnotnull(listing_id#9757)
                          : +- *(2) ColumnarToRow
                          : +- FileScan parquet [id#9756,listing_id#9757] Batched: true, DataFilters: [isnotnull(listing_id#9757)], Forma
t: Parquet, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_reviews_by_year-large.parquet/yr..., PartitionFilters: [], Pu
shedFilters: [IsNotNull(listing_id)], ReadSchema: struct<id:string,listing_id:string>
                          +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string, true]),false), [plan_id=32817]
                          +- *(3) Project [id#9710, city_name#9711]
                          +- *(3) Filter ((isnotnull(country_code#9714) AND (country_code#9714 = AU)) AND isnotnull(id#9710))
                          +- *(3) ColumnarToRow
                          +- FileScan parquet [id#9710,city_name#9711,country_code#9714] Batched: true, DataFilters: [isnotnull(country_co
de#9714), (country_code#9714 = AU), isnotnull(id#9710)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb
_cities.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(country_code), EqualTo(country_code,AU), IsNotNull(id)], ReadSchema: struct<id:str
ing,city_name:string,country_code:string>

```

Appendix B: Task3 Execution Plan

Without Optimization

```
== Physical Plan ==
*(46) Project [neighbourhood_name#1118, average_occupancy_rate#1499, listings_struct#1617, pairs#1697, concat([, concat_ws(, , pairs#1697), ]) AS
listings_str#1703, format_number(average_occupancy_rate#1499, 2) AS avg_occ_rate_str#1709]
+- *(46) Project [neighbourhood_name#1118, average_occupancy_rate#1499, listings_struct#1617, transform(listings_struct#1617, lambdafunction(concat((
lambda x_0#1698.listing_name, , cast(lambda x_0#1698.occupancy_rate as string), )), lambda x_0#1698, false)) AS pairs#1697]
+- *(46) Sort [average_occupancy_rate#1499 DESC NULLS LAST, neighbourhood_name#1118 DESC NULLS LAST], true, 0
+- Exchange rangepartitioning(average_occupancy_rate#1499 DESC NULLS LAST, neighbourhood_name#1118 DESC NULLS LAST, 200),
ENSURE_REQUIREMENTS, [plan_id=6038]
+- *(45) Project [neighbourhood_name#1118, average_occupancy_rate#1499, listings_struct#1617]
+- *(45) BroadcastHashJoin [neighbourhood_name#1118], [neighbourhood_name#1682], Inner, BuildRight, false, false
:- ObjectHashAggregate(keys=[neighbourhood_name#1118], functions=[collect_list(struct(rn, rn#1503, listing_name, listing_name#897,
occupancy_rate, occupancy_rate#1443), 0, 0)], output=[neighbourhood_name#1118, listings_struct#1617])
: +- *(29) Project [neighbourhood_name#1118, listing_name#897, occupancy_rate#1443, rn#1503]
: +- *(29) Project [neighbourhood_name#1118, listing_name#897, occupancy_rate#1443, rn#1503]
: +- *(29) BroadcastHashJoin [neighbourhood_name#1118], [neighbourhood_name#1599], Inner, BuildRight, false, false
: :- *(29) Project [neighbourhood_name#1118, listing_name#897, occupancy_rate#1443, rn#1503]
: : +- *(29) Filter (rn#1503 <= 5)
: : +- *(29) RunningWindowFunction [listing_name#897, neighbourhood_name#1118, occupancy_rate#1443,
row_number() window specification (neighbourhood_name#1118, occupancy_rate#1443 DESC NULLS LAST, specified window frame (RowFrame,
unbounded preceding $(), current row $())) AS rn#1503], [neighbourhood_name#1118], [occupancy_rate#1443 DESC NULLS LAST], false
: : +- *(29) Sort [neighbourhood_name#1118 ASC NULLS FIRST, occupancy_rate#1443 DESC NULLS LAST], false, 0
: : +- Exchange hashpartitioning(neighbourhood_name#1118, 200), ENSURE_REQUIREMENTS, [plan_id=5574]
: : +- *(13) Project [listing_name#897, neighbourhood_name#1118,
least((coalesce(nanvl(occupation_days_sum#1330, null), 0.0) / 365.0), 1.0) AS occupancy_rate#1443]
: : +- *(13) SortMergeJoin [listing_id#1144L], [listing_id#950L], LeftOuter
: : :- Sort [listing_id#1144L ASC NULLS FIRST], false, 0

: : +- Exchange hashpartitioning(listing_id#1144L, 200), ENSURE_REQUIREMENTS, [plan_id=5191]
: : +- *(5) Project [id#896L AS listing_id#1144L, listing_name#897, nhoo#980 AS
neighbourhood_name#1118]
: : +- *(5) SortMergeJoin
[knownfloatingpointnormalized(normalizenanandzero(neighbourhood#904))), [knownfloatingpointnormalized(normalizenanandzero(cast(id#978 as
double)))], Inner
: : :- Sort [knownfloatingpointnormalized(normalizenanandzero(neighbourhood#904))
ASC NULLS FIRST], false, 0
: : +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(neighbourhood#904)), 200), ENSURE_REQUIREMENTS, [plan_id=5179]
: : +- *(3) Project [id#896L, listing_name#897, neighbourhood#904]
: : +- *(3) SortMergeJoin
[knownfloatingpointnormalized(normalizenanandzero(city_id#903))), [knownfloatingpointnormalized(normalizenanandzero(cast(id#869 as double)))], Inner
: : :- Sort [knownfloatingpointnormalized(normalizenanandzero(city_id#903))
ASC NULLS FIRST], false, 0
: : +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(city_id#903)), 200), ENSURE_REQUIREMENTS, [plan_id=5166]
: : +- *(1) Filter (isnotnull(city_id#903) AND
isnotnull(neighbourhood#904))
: : +- FileScan csv
[id#896L,listing_name#897,city_id#903,neighbourhood#904] Batched: false, DataFilters: [isnotnull(city_id#903), isnotnull(neighbourhood#904)], Format: CSV,
Location: InMemoryFileIndex(1 paths)[dbfs/FileStore/tables/airbnb_listings-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(city_id),
IsNotNull(neighbourhood)], ReadSchema: struct<id:bigint,listing_name:string,city_id:double,neighbourhood:double>
: : +- Sort
[knownfloatingpointnormalized(normalizenanandzero(cast(id#869 as double))) ASC NULLS FIRST], false, 0
: : +- Exchange
```

```

hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(cast(id#869 as double))), 200), ENSURE_REQUIREMENTS, [plan_id=5173]
:
:
:
:
: +- *(2) Project [id#869]
:
: +- *(2) Filter ((isnotnull(city_name#870) AND (city_name#870 =
Sydney)) AND isnotnull(id#869))
:
: +- FileScan csv [id#869,city_name#870] Batched: false,
DataFilters: [isnotnull(city_name#870), (city_name#870 = Sydney), isnotnull(id#869)], Format: CSV, Location: InMemoryFileIndex(1
paths)[dbfs:/FileStore/tables/airbnb_cities.csv], PartitionFilters: [], PushedFilters: [IsNotNull(city_name), EqualTo(city_name,Sydney), IsNotNull(id)],
ReadSchema: struct<id:int,city_name:string>
:
: +- Sort [knownfloatingpointnormalized(normalizenanandzero(cast(id#978 as double)))
ASC NULLS FIRST], false, 0
:
: +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(cast(id#978 as double))), 200), ENSURE_REQUIREMENTS, [plan_id=5185]
:
: +- *(4) Filter (isnotnull(id#978) AND isnotnull(nhood_name#980))
: +- FileScan csv [id#978,nhood_name#980] Batched: false, DataFilters:
[isnotnull(id#978), isnotnull(nhood_name#980)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_neighbourhoods.csv],
PartitionFilters: [], PushedFilters: [IsNotNull(id), IsNotNull(nhood_name)], ReadSchema: struct<id:int,nhood_name:string>
: +- *(12) Sort [listing_id#950L ASC NULLS FIRST], false, 0
: +- *(12) Project [listing_id#950L, sum(occupation_days#1264)#1329 AS
occupation_days_sum#1330]
: +- *(12) HashAggregate(keys=[listing_id#950L], functions=[finalmerge_sum(merge sum#1724)
AS sum(occupation_days#1264)#1329], output=[listing_id#950L, sum(occupation_days#1264)#1329])
: +- *(12) Project [listing_id#950L, sum#1724]
: +- *(12) SortMergeJoin [listing_id#950L], [listing_id#1144L], Inner
:- Sort [listing_id#950L ASC NULLS FIRST], false, 0
: +- Exchange hashpartitioning(listing_id#950L, 200), ENSURE_REQUIREMENTS,
[plan_id=5198]
:
: +- *(6) Project [listing_id#950L]
: +- *(6) Filter ((year(cast(gettimestamp(review_date#951, yyyy/M/d,
TimestampType, Some(Etc/UTC), false) as date)) = 2024) AND isnotnull(listing_id#950L))
: +- FileScan csv [listing_id#950L,review_date#951] Batched: false,
DataFilters: [(year(cast(gettimestamp(review_date#951, yyyy/M/d, TimestampType, Some(Etc/UTC), false) as date))=, Format: CSV, Location:
InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_reviews-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(listing_id)], ReadSchema:
struct<listing_id:bigint,review_date:date>
: +- Sort [listing_id#1144L ASC NULLS FIRST], false, 0
: +- Exchange hashpartitioning(listing_id#1144L, 200), ENSURE_REQUIREMENTS,
[plan_id=5546]
: +- *(11) HashAggregate(keys=[listing_id#1144L,
functions=[partial_sum(CASE WHEN ((cast(minimum_nights#1338 as int) > 3) AND (cast(minimum_nights#1338 as int) <= 21)) THEN
(cast(minimum_nights#1338 as double) * 2.0) WHEN (cast(minimum_nights#1338 as int) > 21) THEN 6.0 ELSE 6.0 END) AS sum#1724],
output=[listing_id#1144L, sum#1724])
: +- *(11) Project [id#1333L AS listing_id#1144L, minimum_nights#1338]
: +- *(11) SortMergeJoin
[knownfloatingpointnormalized(normalizenanandzero(neighbourhood#1341))), [knownfloatingpointnormalized(normalizenanandzero(cast(id#1356 as
double))), Inner
: :- Sort
[knownfloatingpointnormalized(normalizenanandzero(neighbourhood#1341)) ASC NULLS FIRST], false, 0
: +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(neighbourhood#1341)), 200), ENSURE_REQUIREMENTS, [plan_id=5520]
: +- *(9) Project [id#1333L, minimum_nights#1338,
neighbourhood#1341]
: +- *(9) SortMergeJoin
[knownfloatingpointnormalized(normalizenanandzero(city_id#1340))), [knownfloatingpointnormalized(normalizenanandzero(cast(id#1351 as double))),

```

```

Inner
:
:
:
:- Sort
[knownfloatingpointnormalized(normalizenanandzero(city_id#1340)) ASC NULLS FIRST], false, 0
:
:
: +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(city_id#1340)), 200), ENSURE_REQUIREMENTS, [plan_id=5204]
:
: +- *(7) Filter ((isnotnull(city_id#1340) AND
isnotnull(neighbourhood#1341)) AND isnotnull(id#1333L))
:
: +- FileScan csv
[id#1333L,minimum_nights#1338,city_id#1340,neighbourhood#1341] Batched: false, DataFilters: [isnotnull(city_id#1340), isnotnull(neighbourhood#1341),
isnotnull(id#1333L)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs/FileStore/tables/airbnb_listings-large.csv], PartitionFilters: [], PushedFilters:
[IsNotNull(city_id), IsNotNull(neighbourhood), IsNotNull(id)], ReadSchema: struct<id:bigint,minimum_nights:string,city_id:double,neighbourhood:double>
:
: +- Sort
[knownfloatingpointnormalized(normalizenanandzero(cast(id#1351 as double))) ASC NULLS FIRST], false, 0
:
: +- ReusedExchange [id#1351], Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(cast(id#869 as double))), 200), ENSURE_REQUIREMENTS, [plan_id=5173]
:
: +- Sort
[knownfloatingpointnormalized(normalizenanandzero(cast(id#1356 as double))) ASC NULLS FIRST], false, 0
:
: +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(cast(id#1356 as double))), 200), ENSURE_REQUIREMENTS, [plan_id=5223]
:
: +- *(10) Filter isnotnull(id#1356)
: +- FileScan csv [id#1356] Batched: false, DataFilters:
[isnotnull(id#1356)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs/FileStore/tables/airbnb_neighbourhoods.csv], PartitionFilters: [], PushedFilters:
[IsNotNull(id)], ReadSchema: struct<id:int>
:
: +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string, false]),false), [plan_id=5801]
:
: +- *(28) Filter isnotnull(neighbourhood_name#1599)
:
: +- TakeOrderedAndProject(limit=10, orderBy=[average_occupancy_rate#1499 DESC NULLS LAST],

output=[neighbourhood_name#1599])
:
: +- *(27) HashAggregate(keys=[neighbourhood_name#1599], functions=[finalmerge_avg(merge sum#1730,
count#1731L) AS avg(occupancy_rate#1443)#1498], output=[neighbourhood_name#1599, average_occupancy_rate#1499])
:
: +- Exchange hashpartitioning(neighbourhood_name#1599, 200), ENSURE_REQUIREMENTS, [plan_id=5763]
:
: +- *(26) HashAggregate(keys=[neighbourhood_name#1599], functions=[partial_avg(occupancy_rate#1443)
AS (sum#1730, count#1731L)], output=[neighbourhood_name#1599, sum#1730, count#1731L])
:
: +- *(26) Project [neighbourhood_name#1599, least((coalesce(nanvl(occupation_days_sum#1330, null),
0.0) / 365.0), 1.0) AS occupancy_rate#1443]
:
: +- *(26) SortMergeJoin [listing_id#1144L], [listing_id#1566L], LeftOuter
:
: :- Sort [listing_id#1144L ASC NULLS FIRST], false, 0
:
: +- Exchange hashpartitioning(listing_id#1144L, 200), ENSURE_REQUIREMENTS, [plan_id=5653]
:
: +- *(18) Project [id#1537L AS listing_id#1144L, nhood_name#1562 AS
neighbourhood_name#1599]
:
: +- *(18) SortMergeJoin
[knownfloatingpointnormalized(normalizenanandzero(neighbourhood#1545))), [knownfloatingpointnormalized(normalizenanandzero(cast(id#1560 as
double)))], Inner
:
: :- Sort
[knownfloatingpointnormalized(normalizenanandzero(neighbourhood#1545)) ASC NULLS FIRST], false, 0
:
: +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(neighbourhood#1545)), 200), ENSURE_REQUIREMENTS, [plan_id=5628]
:
: +- *(16) Project [id#1537L, neighbourhood#1545]
:
: +- *(16) SortMergeJoin
[knownfloatingpointnormalized(normalizenanandzero(city_id#1544))), [knownfloatingpointnormalized(normalizenanandzero(cast(id#1555 as double)))],
Inner
:
: :- Sort
[knownfloatingpointnormalized(normalizenanandzero(city_id#1544)) ASC NULLS FIRST], false, 0

```

```

:                                     :                                     :                                     +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(city_id#1544)), 200), ENSURE_REQUIREMENTS, [plan_id=5252]
:                                     :                                     :                                     +- *(14) Filter (isnotnull(city_id#1544) AND
isnotnull(neighbourhood#1545))
:                                     :                                     :                                     +- FileScan csv
[id#1537L,city_id#1544,neighbourhood#1545] Batched: false, DataFilters: [isnotnull(city_id#1544), isnotnull(neighbourhood#1545)], Format: CSV, Location:
InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_listings-large.csv], PartitionFilters: [], PushedFilters: [IsNotNull(city_id), IsNotNull(neighbourhood)],
ReadSchema: struct<id:bigint,city_id:double,neighbourhood:double>
:                                     :                                     :                                     +- Sort
[knownfloatingpointnormalized(normalizenanandzero(cast(id#1555 as double))) ASC NULLS FIRST], false, 0
:                                     :                                     :                                     +- ReusedExchange [id#1555], Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(cast(id#869 as double))), 200), ENSURE_REQUIREMENTS, [plan_id=5173]
:                                     :                                     :                                     +- Sort [knownfloatingpointnormalized(normalizenanandzero(cast(id#1560 as
double))) ASC NULLS FIRST], false, 0
:                                     :                                     :                                     +- Exchange
hashpartitioning(knownfloatingpointnormalized(normalizenanandzero(cast(id#1560 as double))), 200), ENSURE_REQUIREMENTS, [plan_id=5271]
:                                     :                                     :                                     +- *(17) Filter isnotnull(id#1560)
:                                     :                                     :                                     +- FileScan csv [id#1560,nhood_name#1562] Batched: false, DataFilters:
[isnotnull(id#1560)], Format: CSV, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables/airbnb_neighbourhoods.csv], PartitionFilters: [], PushedFilters:
[IsNotNull(id)], ReadSchema: struct<id:int,nhood_name:string>
:                                     :                                     :                                     +- *(25) Sort [listing_id#1566L ASC NULLS FIRST], false, 0
:                                     :                                     :                                     +- *(25) Project [listing_id#1566L, sum(occupation_days#1264)#1329 AS
occupation_days_sum#1330]
:                                     :                                     :                                     +- *(25) HashAggregate(keys=[listing_id#1566L], functions=[finalmerge_sum(merge
sum#1724) AS sum(occupation_days#1264)#1329], output=[listing_id#1566L, sum(occupation_days#1264)#1329])
:                                     :                                     :                                     +- *(25) Project [listing_id#1566L, sum#1724]
:                                     :                                     :                                     +- *(25) SortMergeJoin [listing_id#1566L], [listing_id#1144L], Inner
:                                     :                                     :                                     :- Sort [listing_id#1566L ASC NULLS FIRST], false, 0
:                                     :                                     :                                     :                                     +- ReusedExchange [listing_id#1566L], Exchange
hashpartitioning(listing_id#950L, 200), ENSURE_REQUIREMENTS, [plan_id=5198]
:                                     :                                     :                                     +- Sort [listing_id#1144L ASC NULLS FIRST], false, 0
:                                     :                                     :                                     +- ReusedExchange [listing_id#1144L, sum#1724], Exchange
hashpartitioning(listing_id#1144L, 200), ENSURE_REQUIREMENTS, [plan_id=5546]
+- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string, false]),false), [plan_id=6022]
+- *(44) Filter isnotnull(neighbourhood_name#1682)
+- TakeOrderedAndProject(limit=10, orderBy=[average_occupancy_rate#1499 DESC NULLS LAST],
output=[neighbourhood_name#1682,average_occupancy_rate#1499])
+- *(43) HashAggregate(keys=[neighbourhood_name#1682], functions=[finalmerge_avg(merge sum#1730, count#1731L) AS
avg(occupancy_rate#1443)#1498], output=[neighbourhood_name#1682, average_occupancy_rate#1499])
+- ReusedExchange [neighbourhood_name#1682, sum#1730, count#1731L], Exchange
hashpartitioning(neighbourhood_name#1599, 200), ENSURE_REQUIREMENTS, [plan_id=5763]

```

Physical Optimization

```

== Physical Plan ==
*(22) Project [neighbourhood_name#3345, average_occupancy_rate#3482, listings_struct#3591, pairs#3595, concat([, concat_ws(, , pairs#3595), ],) AS
listings_str#3601, format_number(average_occupancy_rate#3482, 2) AS avg_occ_rate_str#3607]
+- *(22) Project [neighbourhood_name#3345, average_occupancy_rate#3482, listings_struct#3591, transform(listings_struct#3591, lambdafunction(concat([,
lambda x_2#3596.listing_name, , , cast(lambda x_2#3596.occupancy_rate as string), ]), lambda x_2#3596, false))] AS pairs#3595]
+- *(22) Sort [average_occupancy_rate#3482 DESC NULLS LAST, neighbourhood_name#3345 DESC NULLS LAST], true, 0
+- Exchange rangepartitioning(average_occupancy_rate#3482 DESC NULLS LAST, neighbourhood_name#3345 DESC NULLS LAST, 200),
ENSURE_REQUIREMENTS, [plan_id=12117]
+- ObjectHashAggregate(keys=[neighbourhood_name#3345, average_occupancy_rate#3482], functions=[collect_list(struct(rn, rn#3567,
listing_name, listing_name#3241, occupancy_rate, occupancy_rate#3462), 0, 0)], output=[neighbourhood_name#3345, average_occupancy_rate#3482,
listings_struct#3591])
+- *(21) Project [neighbourhood_name#3345, knownfloatingpointnormalized(normalizenanandzero(average_occupancy_rate#3482))] AS
average_occupancy_rate#3482, listing_name#3241, occupancy_rate#3462, rn#3567]
+- *(21) Filter (rn#3567 <= 5)
+- *(21) RunningWindowFunction [neighbourhood_name#3345, listing_name#3241, occupancy_rate#3462,
average_occupancy_rate#3482, row_number() window specification(neighbourhood_name#3345, occupancy_rate#3462 DESC NULLS LAST,
specifiedwindowframe(RowFrame, unboundedpreceding$(), currentrow$())) AS rn#3567], [neighbourhood_name#3345], [occupancy_rate#3462 DESC NULLS
LAST], false
+- *(21) Sort [neighbourhood_name#3345 ASC NULLS FIRST, occupancy_rate#3462 DESC NULLS LAST], false, 0
+- Exchange hashpartitioning(neighbourhood_name#3345, 200), ENSURE_REQUIREMENTS, [plan_id=12076]
+- *(20) Project [neighbourhood_name#3345, listing_name#3241, occupancy_rate#3462, average_occupancy_rate#3482]
+- *(20) BroadcastHashJoin [neighbourhood_name#3345], [neighbourhood_name#3547], Inner, BuildRight, false, false
:- *(20) Project [listing_name#3241, neighbourhood_name#3345, least((coalesce(nanvl(occupation_days_sum#3403,
null), 0.0) / 365.0), 1.0) AS occupancy_rate#3462]
:                                     :                                     :                                     +- *(20) SortMergeJoin [listing_id#3353], [listing_id#3277], LeftOuter
:                                     :                                     :                                     :- Sort [listing_id#3353 ASC NULLS FIRST], false, 0

```

```

:      : +- Exchange hashpartitioning(listing_id#3353, 200), ENSURE_REQUIREMENTS, [plan_id=11691]
:      :      +- *(3) Project [id#3240 AS listing_id#3353, listing_name#3241, nhood_name#3222 AS
neighbourhood_name#3345]
:      :      +- *(3) BroadcastHashJoin [neighbourhood#3248], [id#3220], Inner, BuildRight, false, false
:      :      :- *(3) Project [id#3240, neighbourhood#3248, listing_name#3241]
:      :      +- *(3) BroadcastHashJoin [city_id#3247], [id#3230], Inner, BuildRight, false, false
:      :      :- *(3) Filter (isnotnull(city_id#3247) AND isnotnull(neighbourhood#3248))
:      :      : +- *(3) ColumnarToRow
:      :      : +- FileScan parquet [id#3240,listing_name#3241,city_id#3247,neighbourhood#3248]
Batched: true, DataFilters: [isnotnull(city_id#3247), isnotnull(neighbourhood#3248)], Format: Parquet, Location: InMemoryFileIndex(1
paths)[dbfs/FileStore/tables_parquet/airbnb_listings-large.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(city_id), IsNotNull(neighbourhood)],
ReadSchema: struct<id:string,listing_name:string,city_id:string,neighbourhood:string>
:      :      +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string,
true]),false), [plan_id=11678]
:      :      +- *(1) Project [id#3230]
:      :      +- *(1) Filter (isnotnull(city_name#3231) AND (city_name#3231 = Sydney)) AND
isnotnull(id#3230))
:      :      +- *(1) ColumnarToRow
:      :      +- FileScan parquet [id#3230,city_name#3231] Batched: true, DataFilters:
[isnotnull(city_name#3231), (city_name#3231 = Sydney), isnotnull(id#3230)], Format: Parquet, Location: InMemoryFileIndex(1
paths)[dbfs/FileStore/tables_parquet/airbnb_cities.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(city_name), EqualTo(city_name,Sydney),
IsNotNull(id)], ReadSchema: struct<id:string,city_name:string>
:      :      +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string, false]),false),
[plan_id=11686]
:      :      +- *(2) Filter (isnotnull(id#3220) AND isnotnull(nhood_name#3222))
:      :      +- *(2) ColumnarToRow
:      :      +- FileScan parquet [id#3220,nhood_name#3222] Batched: true, DataFilters:
[isnotnull(id#3220), isnotnull(nhood_name#3222)], Format: Parquet, Location: InMemoryFileIndex(1
paths)[dbfs/FileStore/tables_parquet/airbnb_neighbourhoods.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(id), IsNotNull(nhood_name)],
ReadSchema: struct<id:string,nhood_name:string>
:      :      +- *(8) Sort [listing_id#3277 ASC NULLS FIRST], false, 0
:      :      +- *(8) Project [listing_id#3277, sum(occupation_days#3383)#3402 AS occupation_days_sum#3403]
:      :      +- *(8) HashAggregate(keys=[listing_id#3277], functions=[finalmerge_sum(merge_sum#3642) AS
sum(occupation_days#3383)#3402], output=[listing_id#3277, sum(occupation_days#3383)#3402])
:      :      +- *(8) Project [listing_id#3277, sum#3642]
:      :      +- *(8) SortMergeJoin [listing_id#3277], [listing_id#3353], Inner
:      :      :- Sort [listing_id#3277 ASC NULLS FIRST], false, 0
:      :      : +- Exchange hashpartitioning(listing_id#3277, 200), ENSURE_REQUIREMENTS,
[plan_id=11697]
:      :      : +- *(4) Filter isnotnull(listing_id#3277)
:      :      : +- InMemoryTableScan [listing_id#3277], [isnotnull(listing_id#3277)], false
:      :      :      +- InMemoryRelation [listing_id#3277], StorageLevel(disk, memory,
deserialized, 1 replicas)
:      :      :      +- *(1) Project [listing_id#3277]
:      :      :      +- *(1) Filter (isnotnull(review_date#3278) AND
(year(review_date#3278) = 2024))
:      :      :      +- *(1) ColumnarToRow
:      :      :      +- FileScan parquet [listing_id#3277,review_date#3278]
Batched: true, DataFilters: [isnotnull(review_date#3278), (year(review_date#3278) = 2024)], Format: Parquet, Location: InMemoryFileIndex(1
paths)[dbfs/FileStore/tables_parquet/airbnb_reviews_by_year-large.parquet/yr..., PartitionFilters: [], PushedFilters: [IsNotNull(review_date)], ReadSchema:
struct<listing_id:string,review_date:date>
:      :      +- Sort [listing_id#3353 ASC NULLS FIRST], false, 0

```

```

:                                     +- Exchange hashpartitioning(listing_id#3353, 200), ENSURE_REQUIREMENTS,
[plan_id=11886]
:                                     +- *(7) HashAggregate(keys=[listing_id#3353], functions=[partial_sum(CASE WHEN
((cast(minimum_nights#3411 as int) > 3) AND (cast(minimum_nights#3411 as int) <= 21)) THEN (cast(minimum_nights#3411 as double) * 2.0) WHEN
(cast(minimum_nights#3411 as int) > 21) THEN 6.0 ELSE 6.0 END) AS sum#3642], output=[listing_id#3353, sum#3642])
:                                     +- *(7) Project [id#3406 AS listing_id#3353, minimum_nights#3411]
:                                     +- *(7) BroadcastHashJoin [neighbourhood#3414], [id#3429], Inner, BuildRight,
false, false
:                                     :- *(7) Project [id#3406, neighbourhood#3414, minimum_nights#3411]
:                                     : +- *(7) BroadcastHashJoin [city_id#3413], [id#3424], Inner, BuildRight,
false, false
:                                     :                                     :- *(7) Filter ((isnotnull(city_id#3413) AND
isnotnull(neighbourhood#3414)) AND isnotnull(id#3406))
:                                     :                                     +- *(7) ColumnarToRow
:                                     :                                     +- FileScan parquet
[id#3406,minimum_nights#3411,city_id#3413,neighbourhood#3414] Batched: true, DataFilters: [isnotnull(city_id#3413), isnotnull(neighbourhood#3414),
isnotnull(id#3406)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[dbfs/FileStore/tables_parquet/airbnb_listings-large.parquet], PartitionFilters: [],
PushedFilters: [IsNotNull(city_id), IsNotNull(neighbourhood), IsNotNull(id)], ReadSchema:
struct<id:string,minimum_nights:string,city_id:string,neighbourhood:string>
:                                     :                                     +- ReusedExchange [id#3424], BroadcastExchange
HashedRelationBroadcastMode(ArrayBuffer(input[0, string, true]),false), [plan_id=11678]
:                                     +- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0,
string, false]),false), [plan_id=11716]
:                                     +- *(6) Filter isnotnull(id#3429)
:                                     +- *(6) ColumnarToRow
:                                     +- FileScan parquet [id#3429] Batched: true, DataFilters:
[isnotnull(id#3429)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[dbfs/FileStore/tables_parquet/airbnb_neighbourhoods.parquet], PartitionFilters:
[], PushedFilters: [IsNotNull(id)], ReadSchema: struct<id:string>
+- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0, string, false]),false), [plan_id=12060]
+- *(19) Filter isnotnull(neighbourhood_name#3547)
+- TakeOrderedAndProject(limit=10, orderBy=[average_occupancy_rate#3482 DESC NULLS LAST],
output=[neighbourhood_name#3547,average_occupancy_rate#3482])
+- *(18) HashAggregate(keys=[neighbourhood_name#3547], functions=[finalmerge_avg(merge
sum#3648, count#3649L) AS avg(occupancy_rate#3462)#3481], output=[neighbourhood_name#3547, average_occupancy_rate#3482])
+- Exchange hashpartitioning(neighbourhood_name#3547, 200), ENSURE_REQUIREMENTS,
[plan_id=12022]
+- *(17) HashAggregate(keys=[neighbourhood_name#3547],
functions=[partial_avg(occupancy_rate#3462) AS (sum#3648, count#3649L)], output=[neighbourhood_name#3547, sum#3648, count#3649L])
+- *(17) Project [neighbourhood_name#3547, least((coalesce(nanvl(occupation_days_sum#3403,
null), 0.0) / 365.0), 1.0) AS occupancy_rate#3462]
+- *(17) SortMergeJoin [listing_id#3353], [listing_id#3514], LeftOuter
:- Sort [listing_id#3353 ASC NULLS FIRST], false, 0
: +- Exchange hashpartitioning(listing_id#3353, 200), ENSURE_REQUIREMENTS,
[plan_id=11938]
: +- *(11) Project [id#3485 AS listing_id#3353, nhood_name#3510 AS
neighbourhood_name#3547]
: +- *(11) BroadcastHashJoin [neighbourhood#3493], [id#3508], Inner,
BuildRight, false, false
: :- *(11) Project [id#3485, neighbourhood#3493]
: :- *(11) BroadcastHashJoin [city_id#3492], [id#3503], Inner, BuildRight,
false, false
:                                     :                                     :- *(11) Filter (isnotnull(city_id#3492) AND

```

```

isnotnull(neighbourhood#3493))
:
: +- *(11) ColumnarToRow
:
: +- FileScan parquet
[id#3485,city_id#3492,neighbourhood#3493] Batched: true, DataFilters: [isnotnull(city_id#3492), isnotnull(neighbourhood#3493)], Format: Parquet, Location:
InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_listings-large.parquet], PartitionFilters: [], PushedFilters: [IsNotNull(city_id),
IsNotNull(neighbourhood)], ReadSchema: struct<id:string,city_id:string,neighbourhood:string>
:
: +- ReusedExchange [id#3503], BroadcastExchange
HashedRelationBroadcastMode(ArrayBuffer(input[0, string, true]),false), [plan_id=11678]
:
+- BroadcastExchange HashedRelationBroadcastMode(ArrayBuffer(input[0,
string, false]),false), [plan_id=11750]
:
+- *(10) Filter isnotnull(id#3508)
:
+- *(10) ColumnarToRow
:
+- FileScan parquet [id#3508,nhood_name#3510] Batched: true,
DataFilters: [isnotnull(id#3508)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_neighbourhoods.parquet],
PartitionFilters: [], PushedFilters: [IsNotNull(id)], ReadSchema: struct<id:string,nhood_name:string>
+- *(16) Sort [listing_id#3514 ASC NULLS FIRST], false, 0
+- *(16) Project [listing_id#3514, sum(occupation_days#3383)#3402 AS
occupation_days_sum#3403]
+- *(16) HashAggregate(keys=[listing_id#3514],
functions=[finalmerge_sum(merge sum#3642) AS sum(occupation_days#3383)#3402], output=[listing_id#3514, sum(occupation_days#3383)#3402])
+- *(16) Project [listing_id#3514, sum#3642]
+- *(16) SortMergeJoin [listing_id#3514], [listing_id#3353], Inner
:- Sort [listing_id#3514 ASC NULLS FIRST], false, 0
:
+- Exchange hashpartitioning(listing_id#3514, 200),
ENSURE_REQUIREMENTS, [plan_id=11761]
:
+- *(12) Filter isnotnull(listing_id#3514)

:
+- InMemoryTableScan [listing_id#3514],
[isnotnull(listing_id#3514)], false
:
+- InMemoryRelation [listing_id#3514],
StorageLevel(disk, memory, deserialized, 1 replicas)
:
+- *(1) Project [listing_id#3277]
:
+- *(1) Filter (isnotnull(review_date#3278)
AND (year(review_date#3278) = 2024))
:
+- *(1) ColumnarToRow
:
+- FileScan parquet
[listing_id#3277,review_date#3278] Batched: true, DataFilters: [isnotnull(review_date#3278), (year(review_date#3278) = 2024)], Format: Parquet, Location:
InMemoryFileIndex(1 paths)[dbfs:/FileStore/tables_parquet/airbnb_reviews_by_year-large.parquet/yr..., PartitionFilters: [], PushedFilters:
[IsNotNull(review_date)], ReadSchema: struct<listing_id:string,review_date:date>
+- Sort [listing_id#3353 ASC NULLS FIRST], false, 0
+- ReusedExchange [listing_id#3353, sum#3642], Exchange
hashpartitioning(listing_id#3353, 200), ENSURE_REQUIREMENTS, [plan_id=11886]

```